

IA Aplicada com LLMs

Aula 03 — O prompt como código: versionado e testado

Onde paramos

Os dois decks anteriores trataram o prompt como algo que você **escreve**.

Este trata dele como algo que **vive em produção** — e que precisa sobreviver ao que todo código de produção enfrenta:

alguém vai editar aquele texto

A pergunta do deck

Como você garante que a edição não quebrou nada?

Em código, a resposta é conhecida: **versionar, revisar, testar**.

O que muda aqui — e é o motivo do deck existir — é que **as três funcionam de forma diferente** quando o artefato é um prompt.

Roteiro

1. Onde o prompt mora hoje
2. Três eixos, uma unidade
3. **Raio de alcance não-local** — por que a suíte é obrigatória
4. **Teste de regressão** — e a demonstração
5. **O que isto tem a ver com evals**

Onde o prompt mora hoje

	Código	Prompt
Onde vive	arquivo versionado	string no meio de um <code>.py</code>
Passa por review	sim	não
Tem teste	sim	não
Qual versão está em produção?	<code>git log</code>	<i>"acho que a Ana mexeu"</i>
Dá para reverter	<code>git revert</code>	—

Não é má-fé: prompt **parece** conteúdo. Está em português, qualquer pessoa lê, e mudar parece inofensivo.

Três eixos, uma unidade

```
comportamento = prompt × modelo × parâmetros
```

└── alguém "ajustou"

└── o alias -latest mudou SOZINHO

└── alguém "melhorou" o texto

Na Aula 02 você viu que `-latest` é **dependência sem versão**.

O prompt é a mesma coisa. Os parâmetros também.

A consequência

Se você não versiona os três juntos, não consegue reproduzir bug nenhum.

O resultado piorou na terça.

Foi o prompt que editaram, o modelo que o provedor atualizou, ou o `top_p` que entrou num PR de "ajuste fino"?

Sem os três registrados, a resposta é **palpite** — e a correção também.

A unidade versionada

```
# prompts/extracao-chamado/v3.yaml
versao: 3
modelo: mistral-small-2506           # versão DATADA, não o alias
parametros:
  temperature: 0
  max_tokens: 120
template: |
  Classifique a mensagem em: {categorias}.
...
```

"Mas o git já versiona o arquivo"

O `git log` responde: *"como este arquivo mudou ao longo do tempo?"*

A pergunta que aparece em produção é outra:

Um cliente reclamou de uma resposta errada dada **terça, 14h20**.

Qual prompt, com qual modelo e quais parâmetros, gerou aquilo?

O git sabe o que o arquivo continha em cada commit.

Não sabe qual commit estava rodando naquela hora.

O número é um nome para a combinação

```
log.info("extracao concluida",  
        extra={"prompt_versao": CONFIG["versao"],    # <- o carimbo  
              "modelo": CONFIG["modelo"],  
              "chamado_id": chamado.id})
```

Com o carimbo no log de cada requisição, a pergunta de terça vira uma **consulta** — e aí sim o `git log` ajuda, porque você sabe o que procurar.

Quando incrementar? Quando o **comportamento** muda.

E o árbitro é a suíte: *resultado diferente = versão nova*.

"Mas isso é só boa prática — eu já versiono tudo"

Falta a parte que é **específica de LLM**:

Em código, você raciocina sobre o impacto de uma mudança.

Num prompt, não dá.

Raio de alcance não-local

Altere uma função: você sabe quem a chama, e delimita o estrago.

Acrescente "**seja conciso**" ao prompt: pode **quebrar a extração de um campo que a frase nem menciona**.

- não existe análise estática de prompt
- não existe "encontrar usos"
- não existe tipo que garanta nada

Só existe medir. É daí que vem a suíte — não de disciplina de processo.

Tirar o prompt do código

```
from pathlib import Path

PROMPTS = Path(__file__).parent / "prompts"

def carregar_prompt(nome: str, **variaveis) -> str:
    texto = (PROMPTS / f"{nome}.md").read_text(encoding="utf-8")
    return texto.format(**variaveis)
```

Dez linhas resolvem 80%: o prompt aparece no `git diff`, entra no review do PR e volta com `git revert`.

Arquivo ou registry?

	Arquivo no repo	Prompt registry
Review	entra no PR	fora do fluxo
Rollback	<code>git revert</code>	pela ferramenta
Mudar sem deploy	não	sim

O registry vende "mudar sem deploy" como vantagem.

É a **mesma** propriedade que faz a mudança escapar do review e do teste.

Comece pelo arquivo.

Teste de regressão

```
CASOS = [  
    ("Meu pedido 48219 não chegou", "entrega_atrasada", "48219"),  
    ("você entregam no sábado?", "duvida", None),  
]  
  
def test_extracao():  
    for mensagem, categoria, pedido in CASOS:  
        saida = extrair(mensagem)  
        assert saida["categoria"] == categoria  
        assert saida["pedido"] == pedido
```

Repare no que **não** é testado: o texto exato da resposta.

O teste é flaky por natureza

`temperature=0` **não garante saída idêntica** (Aula 02, nota 02, §6).

O mesmo teste pode passar hoje e falhar amanhã **sem nada ter mudado**.

Isso não é defeito da suíte — é propriedade do sistema sob teste.

```
def test_suite():  
    media = sum(taxa_de_acerto(c) for c in CASOS) / len(CASOS)  
    assert media >= 0.90, f"regressão: {media:.0%}"
```

O critério vira **k de N**, não "passou".

Duas decisões de projeto

O limiar (90%) tem que ser definido **antes** de ver o resultado.

Senão você ajusta o limiar até passar — e o teste não testa nada.

N não sai de graça. $N=5$ sobre 20 casos = 100 chamadas por execução, e cada uma leva alguns segundos. Suíte completa antes de mergear; versão reduzida durante o desenvolvimento.

A demonstração

Prompt v1:

Extraia o número do pedido. Responda apenas o número.

8/8

Um PR de "limpeza de texto"

Prompt v2:

```
Extraia o número do pedido. Responda o número.
```

5/8

Três casos passaram a vir assim:

```
O número do pedido é 48219.
```

A palavra "apenas" era o contrato inteiro

Nenhum revisor humano teria apontado isso num diff **de uma palavra**.

A suíte apontou em **quatro segundos**.

Revisar prompt lendo o texto é como revisar código sem rodar os testes.

Funciona até não funcionar — e você descobre em produção.

Isto é o embrião do eval

	Regressão (hoje)	Eval (fim do curso)
Tamanho	5–20 casos	50–500
Critério	propriedade objetiva	+ LLM-as-judge, rubrica
Quando roda	antes de mergear	+ contínuo, em produção
Pergunta	<i>"quebrei algo?"</i>	<i>"é bom o suficiente?"</i>

Escrever a versão pequena agora **não duplica** a aula de evals: **prepara**.

Síntese

- **Prompt determina comportamento como código** — mas vive sem versão, review, teste ou rollback
- **comportamento = prompt × modelo × parâmetros** . Versionar um sem os outros **impede reproduzir bug**. Em produção, versão **datada**
- A justificativa da suíte é o **raio de alcance não-local**: não há análise estática de prompt. **Só dá para medir**
- **Dez linhas** tiram o prompt do código e já entregam diff, review e `git revert` . Registry é decisão de organização
- **Teste por propriedade**, e **flaky por natureza**: critério **k de N**, com o limiar definido **antes** de ver o resultado
- Uma palavra removida: **8/8** → **5/8**. Revisar prompt lendo o texto é como revisar código sem rodar os testes